

Testing with the demo repo

The demo repo is DemoPay, a fake payments app with secrets and decoys planted in it. A **decoy** looks like a secret but is not one, such as a placeholder or a commit hash. We planted everything ourselves, so we know the right answer for every line, and we can measure how well SecureGate does.

The three files

- `src/securegate/demo/catalog.yaml`: the list of things to plant. **This is the file you edit.**
- `../securegate-demo/`: the demo repo that `make demo` builds. Don't edit it by hand; it is rebuilt every time.
- `../securegate-demo/ground_truth.csv`: the answer sheet. It lists every planted line and what SecureGate should decide. It never contains the values themselves.

Add a planted secret

Example: a GitHub token pasted into a deploy script. Open `catalog.yaml` and add this item at the end of the `items` list:

```
- kind: github_pat
  file: scripts/deploy.sh
  placement: current
  expected: block
  note: Token pasted into a deploy script.
```

- `kind`: what to plant. All kinds are listed at the bottom of `catalog.yaml`.
- `file`: where. A new file is created for you.
- `placement`: `current` keeps it in the latest code. `history-only` adds it in an early commit and deletes it later; that tests history scanning. A history-only item needs a file of its own.
- `expected`: the decision SecureGate should make: `block`, `warn` or `ignore`.

Add a decoy

The same, with a decoy kind and usually `expected: ignore`:

```
- kind: uuid
  file: services/billing.yaml
  placement: current
  expected: ignore
  name: BILLING_API_CLIENT_ID
```

`name` is optional. It sets the variable name written next to the value. Names containing words like KEY, TOKEN or API make a decoy harder, because scanners pay attention to them.

Rebuild and scan

```
make demo
make scan-demo
```

Your new lines appear in the table (masked), for example `scripts/deploy.sh:4 ghp_****uvW4`.

If `catalog.yaml` has a mistake, `make demo` stops and says what is wrong:

```
securegate: error: ...catalog.yaml: item #19: 'kind' must be one of: aws_key_pair,
stripe_live, github_pat, ... (did you mean 'github_pat'?)
```

Read the scorecard

```
make test
```

At the end, the tests print a scorecard for the demo repo: once for `repo` mode (the whole history) and once for `dir` mode (only the files on disk). With the two examples above added:

```
repo mode, seed 42
planted lines: 22 (10 secret, 12 decoy)
  hits                16
  misses              2
    db_url_password at config/app.yaml:7: expected warn, got nothing
    generic_password at Dockerfile:6: expected warn, got nothing
wrong decisions  1
  aws_key_pair at config/settings.py:12: expected block, got warn
false alarms    3
  uuid at config/app.yaml:8: expected ignore, got warn
  git_sha at web/version.js:2: expected ignore, got warn
  uuid at services/billing.yaml:2: expected ignore, got warn
```

Word	Meaning
hit	SecureGate decided what you expected. For a decoy: it stayed silent or ignored it.
miss	A secret SecureGate did not report: a false negative . The most dangerous mistake.
wrong decision	Found, but blocked instead of warned, or the other way round.
false alarm	A decoy reported as block or warn: a false positive . Too many, and people stop trusting the tool.

When TruffleHog, Semgrep and Bandit are installed (`make scanners`), more scorecards follow: one for all four scanners together, and one for each scanner alone, so you can see what each of them adds. TruffleHog runs there without asking any provider whether a key works, so the fake keys are never sent anywhere.

The numbers are what really happens. We don't change the rules just to make them look better. A miss is a finding about the scanner: write it down in Decisions (`decisions.pdf`).

Test the demo pull requests

`tests/test_demo_kit.py` checks `securegate demo-pr`, `demo-cleanup`, `doctor` and `ci-report` without touching GitHub: a real `git` works on a throwaway repository whose "GitHub" is a folder next to it, and a stand-in for `gh` records every command. The tests prove that a demo pull request is built from origin/main, pushes exactly one new `demo/` branch and never main, refuses a folder with uncommitted changes, and never prints a fake value; that clean-up only touches `demo/` branches, never a fork's; and that each scene gets the decision it promises, from the policy and from the real Gitleaks.

On GitHub itself, run the five scenes once: Demo: the merge gate (`demo-script.pdf`). `make doctor` first says whether everything is ready.

Test the AI agents

The agents are tested without Azure: a fake model plays scripted answers and records every request it was sent. `tests/test_agents_safety.py` plants values in code and proves none survives redaction, including for every finding of the demo project; `tests/test_agents.py` runs all three agents on the demo project and proves that nothing planted appears in anything that would be sent to Azure, that an instruction hidden in a code comment stays data, and that Azure being down changes nothing but the advice.

`tests/test_agents_ui.py` covers the comment, the dashboard, the setup and the connection check. No test ever calls Azure: an automatic fixture makes the agents count as not set up unless a test passes its own fake model.

Same seed, same repo

`make demo` always uses seed 42, a number that fixes every random choice. The same catalog and seed always build exactly the same repo, so results can be compared over time. To try other values:

```
.venv\Scripts\securegate demo-repo --out ..\securegate-demo --seed 7 --force
```